

SIMATS ENGINEERING

ASSESSMENT TOOL 1

Name : SHESHATHRI S

Reg no : 1925260316

COURSE Name&Code : Artificial Intelligence COURSE CODE &
CSA1707

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to

solve problems effectively. (BL3)

Assessment Tool Weightage: 35%

Annexure A

Analytical Problem Solving

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one.

Neither has any measuring maker on it. There is a pump that can be used to fill the

jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug?

Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are

no other measuring devices available.

Analytical Problem Solving

1. Water Jug Problem

Problem Understanding

We are given two unmarked water jugs:

- Jug A = 4 gallons
- Jug B = 3 gallons
- There is a water pump.
- We can fill a jug completely.
- We can empty a jug onto the ground.
- We can pour water from one jug into another.
- No other measuring device is available.

Goal: Obtain exactly **2 gallons of water in the 4-gallon jug.**

Method Selection

This can be solved as a **state-space search problem.**

Represent a state as:

(x, y)

where:

- x = amount of water in the 4-gallon jug
- y = amount of water in the 3-gallon jug

Initial state:

(0, 0)

Goal state:

(2, 0) or any state where the 4-gallon jug contains exactly 2 gallons.

Step-by-Step Solution

Step	Operation	4-Gallon Jug	3-Gallon Jug

1	Start	0	0
2	Fill 3-gallon jug	0	3
3	Pour 3-gallon jug into 4-gallon jug	3	0
4	Fill 3-gallon jug again	3	3
5	Pour from 3-gallon into 4-gallon until 4-gallon is full	4	2
6	Empty 4-gallon jug	0	2
7	Pour remaining 2 gallons into 4-gallon jug	2	0

State Sequence

$(0,0) \rightarrow (0,3) \rightarrow (3,0) \rightarrow (3,3) \rightarrow (4,2) \rightarrow (0,2) \rightarrow (2,0)$

Final Answer

The 4-gallon jug contains exactly:

2 gallons

The 3-gallon jug contains:

0 gallons

Interpretation

The required quantity is obtained without any measuring device. The solution demonstrates how a real-world problem can be represented as states and solved systematically using state-space search.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

- i. What are the percept' s for this agent?
- ii. Characterize the operating environment.
- iii. What are the actions the agent can take?
- iv. How can one evaluate the performance of the agent?
- v. What sort of agent architecture do you think is most suitable for this agent?

Why?

Mars Rover – Intelligent Agent Analysis

A Mars Rover is an autonomous intelligent agent that observes the Martian environment, makes decisions, moves around the surface, and analyzes collected samples.

i. Percepts of the Agent

The rover receives information from its sensors.

Important percepts include:

- Images and video of the Martian surface
- Distance from obstacles
- Terrain condition
- Rock and soil characteristics
- Temperature
- Atmospheric conditions

- Rover position and orientation
- Battery/power level
- Sample characteristics
- Communication signals from Earth

Thus, the rover uses sensors to continuously obtain information about its surroundings and internal condition.

ii. Characterization of the Operating Environment

The Mars Rover environment can be characterized as follows:

Property	Mars Rover Environment
Observability	Partially observable
Determinism	Stochastic/uncertain
Episodic/Sequential	Sequential
Static/Dynamic	Dynamic
Discrete/Continuous	Mostly continuous
Known/Unknown	Partially unknown
Number of agents	Primarily a single-agent environment

Explanation

- **Partially observable:** The rover cannot see or sense the entire Martian environment.
- **Uncertain:** Wheel movement, terrain, weather and sensor readings may not behave exactly as expected.
- **Sequential:** A current action affects future decisions.

- **Dynamic:** Environmental conditions can change.
- **Continuous:** Position, distance, speed and other measurements vary continuously.
- **Partially unknown:** The rover explores areas that have not been completely mapped or understood.

iii. Actions the Agent Can Take

The rover can perform several actions:

1. Move forward.
2. Move backward.
3. Turn left.
4. Turn right.
5. Stop.
6. Avoid obstacles.
7. Navigate to a selected location.
8. Capture images.
9. Identify rocks or soil.
10. Collect samples.
11. Analyze samples using scientific instruments.
12. Store collected data.
13. Transmit information to Earth.
14. Manage battery/power.
15. Change its exploration path based on sensor information.

iv. Performance Evaluation

The rover's performance can be evaluated using measurable criteria such as:

- Number of useful samples collected.
- Accuracy of sample identification and analysis.
- Distance successfully explored.
- Number of scientific observations completed.
- Accuracy of navigation.
- Number of obstacles successfully avoided.
- Amount of useful data transmitted.
- Energy/battery efficiency.
- Mission time.
- Safety and avoidance of rover damage.

Performance Measure

A good rover should **maximize useful scientific information and successful exploration** while minimizing energy consumption, travel time, navigation errors and risk of damage.

v. Suitable Agent Architecture

Suitable Architecture: Learning Agent with Utility-Based Decision Making

A **learning agent** is suitable because the rover operates in an environment where complete information is not known beforehand.

The rover can:

- Learn from sensor observations.
- Improve navigation decisions.
- Learn about terrain and obstacles.
- Adapt its exploration strategy.
- Use previous experiences to make better decisions.

A utility-based component can help the rover choose actions by considering factors such as:

scientific value + safety + energy consumption + distance + mission objectives.

Why this architecture is suitable

Mars is an uncertain and partially observable environment. Therefore, the rover needs to perceive its surroundings, maintain useful knowledge, make decisions, learn from experience, and select actions that provide useful results while conserving resources.

Interpretation

The Mars Rover is a strong example of an intelligent agent because it follows the basic agent cycle:

Perceive → Analyze → Decide → Act → Learn

3. Explore the search space using search strategies and formulate the problem

components for the 8-Queens problem using the following information: Place 8

queens on a chessboard such that none of the queen's attack any of the others. A

configuration of 8 queens on the board as shown in the figure below, but this does not

represent a solution as the queen in the first column is on the same diagonal as the

queen in the last column.

3. 8-Queens Problem

Problem Understanding

The objective is to place 8 queens on an 8×8 chessboard such that no two queens attack each other.

A queen can attack another queen if they are:

- In the same row.
- In the same column.
- On the same diagonal.

Therefore, a valid solution must satisfy all three conditions.

Problem Formulation

Initial State

An empty 8×8 chessboard.

State

A state represents a partial or complete placement of queens.

One convenient representation is:

$[r_1, r_2, r_3, \dots, r_n]$

where r_i represents the row occupied by the queen in column i .

For example:

$[1, 5, 8, 6, 3, 7, 2, 4]$

represents one queen in each column.

Actions

Place a queen in a row of the next empty column, provided that it does not conflict with already placed queens.

Transition Model

Adding one valid queen to the next column produces a new state.

Goal Test

The goal is reached when:

- 8 queens are placed.
- No two queens share the same row.
- No two queens share the same column.
- No two queens share a diagonal.

Path Cost

For the standard 8-Queens formulation, each valid queen placement can be considered to have a cost of 1.

Thus, a complete solution has a placement cost of:

Search Strategy

A suitable method is Depth-First Search (DFS) / Backtracking.

The search places queens column by column. If a placement causes a conflict, that branch is abandoned and the algorithm goes back to the previous column.

Backtracking Process

1. Start with column 1.
2. Place a queen in a safe row.
3. Move to the next column.
4. Check whether the new queen conflicts with existing queens.
5. If safe, continue.
6. If no safe position exists, backtrack to the previous column.
7. Change the previous queen's position.
8. Continue until all 8 queens are placed.

One Valid Solution

A valid solution is:

[1, 5, 8, 6, 3, 7, 2, 4]

That means:

Column Row

1	1
2	5

3	8
4	6
5	3
6	7
7	2
8	4

Board Representation

```

Q . . . . .
. . . . Q . .
. . . . . Q
. . . Q . . .
. Q . . . . .
. . . . . Q .
. . . . . Q .
. . Q . . . .

```

Here, every column contains exactly one queen, every row contains exactly one queen, and no two queens lie on the same diagonal.

Interpretation The 8-Queens problem demonstrates how search-space exploration and backtracking can solve a constraint-based AI problem. Invalid configurations are rejected early, reducing unnecessary exploration.

4. A customer wants to travel from the one location to another location using OLA Cab

booking mobile application. The customer can select the any of the cab type as mini,

micro, sedan, shared, prime and so on based on his comfort to reach the destination.

Identify what type of problem-solving agent can be used and write the pseudocode

for the above problem.

4. OLA Cab Booking – Problem-Solving Agent

Problem Understanding

The customer wants to travel from a source location to a destination using a cab-booking application.

Available cab types may include:

- Mini
- Micro
- Sedan
- Shared
- Prime

The customer chooses a cab based on requirements such as comfort, cost, availability and travel requirements.

Type of Problem-Solving Agent

A suitable agent is a **Utility-Based Agent**.

Reason

A utility-based agent compares possible choices and selects an action based on how useful each option is.

For example, the customer may want to balance:

- Comfort
- Cost
- Travel time
- Availability
- Distance
- Cab type

Therefore, instead of simply finding any cab, the system can evaluate the available alternatives and select the option that provides the highest utility according to the customer's preferences.

Problem Formulation

Initial State

Customer is at the source location and wants to reach the destination.

Goal State

Customer reaches the selected destination.

Actions

- Enter source.
- Enter destination.
- Select cab preferences.
- Search available cabs.
- Compare cab options.
- Select a cab.
- Book the cab.

- Travel to destination.

Performance Measure

The selected option should satisfy the customer's requirements while considering:

comfort + cost + time + availability.

Pseudocode

BEGIN

INPUT source_location

INPUT destination_location

INPUT customer_preferences

 comfort

 budget

 preferred_cab_type

 maximum_wait_time

REQUEST available_cabs

IF no cabs are available THEN

 DISPLAY "No cabs available"

STOP

END IF

FOR each available cab

 Calculate estimated_fare

 Calculate estimated_wait_time

 Check cab_type

 Calculate comfort_score

 Calculate utility_score based on:

 comfort

 fare

 waiting_time

 cab_type

 customer_preferences

END FOR

SELECT cab with suitable/highest utility score

DISPLAY:

 cab type

 estimated fare

 estimated waiting time

 estimated travel time

IF customer confirms booking THEN

 BOOK selected cab

 DISPLAY booking confirmation

ELSE

 DISPLAY "Booking cancelled"

END IF

END

Interpretation

The utility-based approach allows the application to consider multiple factors instead of selecting a cab using only one criterion. This makes the decision more suitable for a customer with different preferences.

5. A logistics company operates a delivery network where packages must be transported

between warehouses at minimum cost. Each route between warehouses has an

associated travel cost (fuel + time). The company wants to determine the least-cost

path from the starting warehouse S to the destination warehouse G using the Uniform

Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:

5. Uniform Cost Search (UCS) – Logistics Delivery Network

Problem Understanding

The logistics company has a weighted network of warehouses.

- **S** = Starting warehouse
- **G** = Goal/destination warehouse
- Each edge represents a route.
- Each edge has a travel cost based on factors such as fuel and time.
- The objective is to find the **minimum-cost path from S to G**.

The appropriate algorithm is **Uniform Cost Search (UCS)**.

The question specifically asks for the least-cost path using UCS.

Method Selection – Uniform Cost Search

UCS always expands the currently discovered path having the **lowest cumulative path cost**.

If a path is:

$S \rightarrow A \rightarrow B \rightarrow G$

and the edge costs are:

- $S \rightarrow A = c_1$
- $A \rightarrow B = c_2$
- $B \rightarrow G = c_3$

then the total path cost is:

$c_1 + c_2 + c_3$

UCS compares cumulative costs of alternative paths and expands the least-cost path first.

UCS Procedure

1. Start from node **S** with cost 0.
2. Insert S into the priority queue.
3. Select the node with the smallest cumulative cost.
4. Expand that node.
5. Calculate the cumulative cost of each successor.
6. Add the successors to the priority queue.
7. Again select the node with the smallest cumulative cost.
8. Continue this process.
9. When **G** is removed from the priority queue for expansion/selection, the corresponding path is the least-cost path, assuming non-negative edge costs.

UCS Pseudocode

UCS(Graph, Start, Goal)

Create priority queue PQ

Insert Start into PQ with path cost 0

Create cost table

cost[Start] = 0

WHILE PQ is not empty

Remove node N having the lowest path cost

IF N = Goal THEN

RETURN path to N and total cost

END IF

FOR each successor M of N

$\text{new_cost} = \text{cost}[N] + \text{edge_cost}(N, M)$

IF M is not visited

OR $\text{new_cost} < \text{cost}[M]$ THEN

$\text{cost}[M] = \text{new_cost}$

Store parent of M as N

Insert M into PQ with priority new_cost

END IF

END FOR

END WHILE

RETURN "No path exists"

END

Important Calculation

For every possible route:

Total Cost = Sum of all edge costs on that route

For example:

$S \rightarrow A \rightarrow G$

$\text{Cost} = (S,A) + (A,G)$

$S \rightarrow B \rightarrow G$

$\text{Cost} = (S,B) + (B,G)$

$S \rightarrow A \rightarrow C \rightarrow G$

$\text{Cost} = (S,A) + (A,C) + (C,G)$

UCS compares these cumulative costs and selects the path with the smallest value.

Final Result

The **least-cost path** is the path from S to G with the minimum **cumulative edge cost**, found by repeatedly expanding the lowest-cost frontier node.

